

UFUG 2602

Stacks & Queues

Tengfei Chang, Jia Li, Xihu Zheng, Zijun Gong

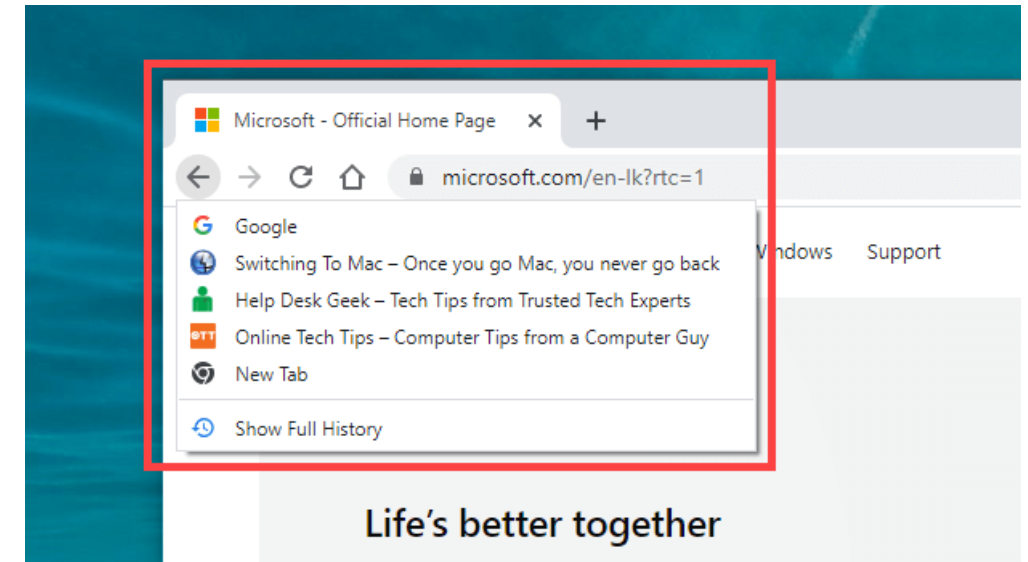
IoT, DSA, INTR Thrusts
Hong Kong University of Science and Technology (Guangzhou)

Outline

- Stacks
 - **Stack Definition**
 - Stack Implementation (Array / Linked-list)
 - Matching Parenthesis
- Queues
 - Queue Definition
 - Queue Implementation

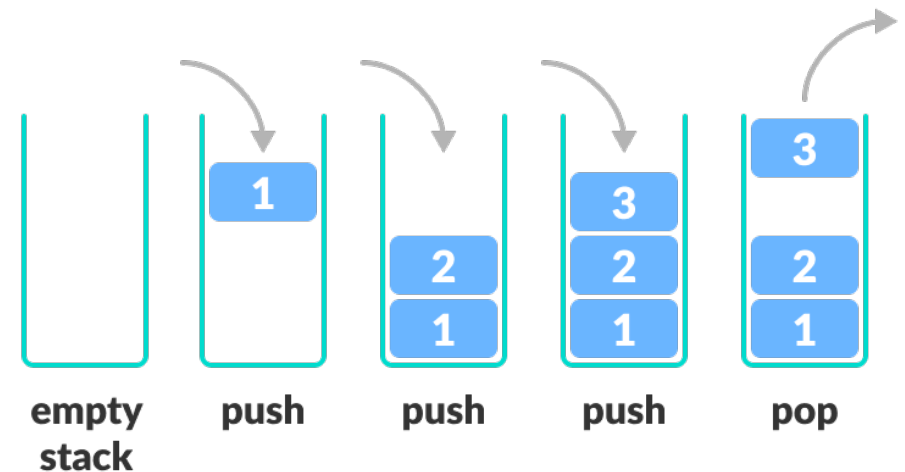
Stacks - Example

- Imagine you're browsing the internet and clicking on different web pages. If you want to go back to a page you visited earlier, you can either:
 - Type the URL of the previous page in the address bar.
 - Click the "Back" button to return through the pages you've seen.
- The "Back" function helps you retrace your steps in the same order you visited the pages.
- This "**Last-In, First-Out**" (LIFO) principle is what defines the Stack data structure!



Stacks

- A **stack** manages a set S of elements and supports two operations:
 - push(e): insert a new element e into S
 - pop: remove the most recently inserted element from S and returns it (at the so-called “top” of the stack).
- Elements are inserted and removed according to the **last-in, first-out (LIFO)** principle.



The Stack Abstract Data Type

- **A stack is a collection that supports two core operations:**
 - **S.push(e):** Add element e to the top of the stack.
 - **S.pop():** Remove and return the top element; raises an error if the stack is empty.
- **Convenient Methods:**
 - **S.top():** Return the top element without removing it.
 - **S.is_empty():** Return True if the stack is empty.
 - **len(S):** Return the number of elements in the stack.
- **Key Properties:**
 - A stack is initially empty.
 - It has no fixed capacity.
 - Elements can be of **any type**.

Operation	Return Value	Stack Contents
S.push(5)	–	[5]
S.push(3)	–	[5, 3]
len(S)	2	[5, 3]
S.pop()	3	[5]
S.is_empty()	False	[5]
S.pop()	5	[]
S.is_empty()	True	[]
S.pop()	“error”	[]
S.push(7)	–	[7]
S.push(9)	–	[7, 9]
S.top()	9	[7, 9]
S.push(4)	–	[7, 9, 4]
len(S)	3	[7, 9, 4]
S.pop()	4	[7, 9]
S.push(6)	–	[7, 9, 6]
S.push(8)	–	[7, 9, 6, 8]
S.pop()	8	[7, 9, 6]

A series of stack operations and their effect on an initially empty stack S.

Exercise 1 — Illegal operation

- Start with an empty stack. Execute:

```
push (5)
push (3)
pop ()
push (2)
pop ()
pop ()
pop ()
```

Questions

1. Show the stack after each operation.
2. What happens during the last `pop ()` and why?

Exercise 2 — Trace push/pop sequence

- Start with an empty stack:

```
push (A)
push (B)
push (C)
pop ()
push (D)
pop ()
push (E)
```

Questions

1. What is the final stack (bottom → top)?
2. What elements were returned by `pop ()`?

Outline

- Stacks
 - Stack Definition
 - **Stack Implementation (Array / Linked-list)**
 - Matching Parenthesis
- Queues
 - Queue Definition
 - Queue Implementation

Array-Based Stack Implementation

- A simple way of implementing the Stack ADT uses an array
- We add elements from left to right
- A variable t keeps track of the index of the top element

Algorithm *size()*

return $t + 1$

Algorithm *pop()*

if *isEmpty()* **then**

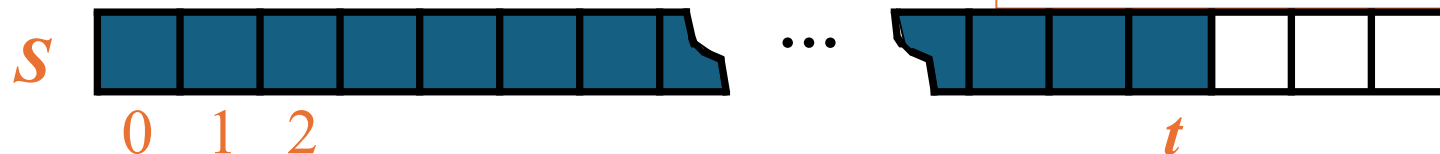
throw *EmptyStackException*

else

$e \leftarrow S[t]$

$t \leftarrow t - 1$

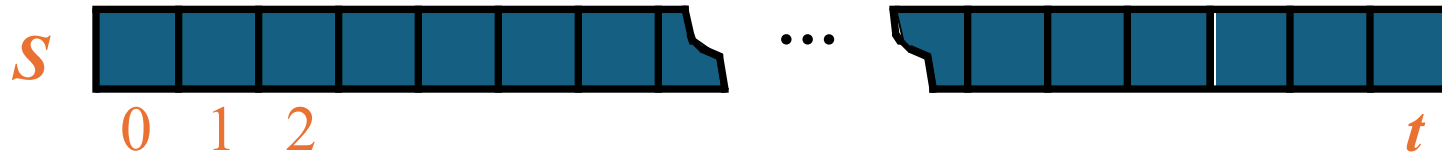
return e



Array-Based Stack Implementation

- The array storing the stack elements may become full
- A push operation will then throw a **FullStackException**
 - Limitation of the array-based implementation
 - Not intrinsic to the Stack ADT
 - Fix this issue by resizing the array

```
Algorithm push(o)  
  if  $t = S.length - 1$  then  
    throw FullStackException  
  else  
     $t \leftarrow t + 1$   
     $S[t] \leftarrow o$ 
```



Array-Based Stack Implementation

- A Stack might be empty
- `top` returns the element at the top of the Stack, but does not remove the top element. When the Stack is empty, it should return none.

```
Algorithm isEmpty()  
    if  $t < 0$  then return true  
    else return false
```

```
Algorithm top()  
    if isEmpty() then  
        throw EmptyStackException  
    return  $S[t]$ 
```



Array-Based Stack Implementation

- **Time Complexity:**
 - **top(), is_empty(), len():** Constant time $O(1)$ in the worst case.
 - **push() and pop():** Amortized $O(1)$, but can take $O(n)$ in the worst case due to internal array resizing.
- **Space Complexity:**
 - The space usage for a stack is $O(n)$.
- **Efficiency Tip:**
 - It is more efficient to initialize the list with length n rather than starting with an empty list and appending items (if we know the maximum size).
 - Both methods use $O(n)$ space, but pre-sizing is faster in practice.

Operation	Running Time
S.push(e)	$O(1)^*$
S.pop()	$O(1)^*$
S.top()	$O(1)$
S.is_empty()	$O(1)$
len(S)	$O(1)$

*amortized

Performance of our array-based stack implementation.

Exercise 3 — Implement Stack ADT (answer)

- A correct Python-list implementation:

Exercise 4 — Array stack resizing

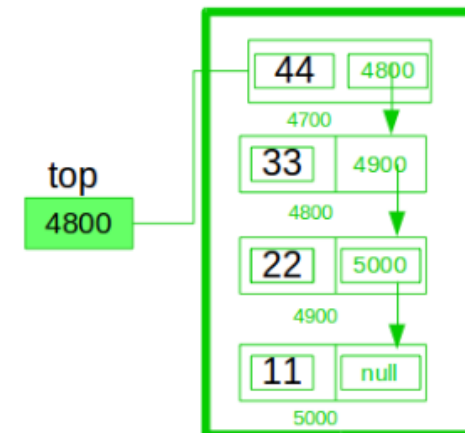
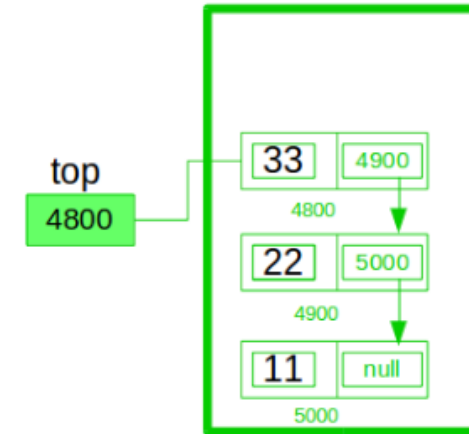
- Assume stack starts with capacity = 1.
When full $\rightarrow$ capacity doubles.
- Perform:
 push 1...8

Questions

1. What are the capacities after each resize?
2. How many resizes happen?

Linked List-Based Implementation

- Using a singly linked list for a stack:
 - Store the elements of S in a singly linked list *L*.
 - At all times, keep track of a **top pointer** to the first node
 - Pushing and popping items happens **at the head of the list**.
- Why Linked List?
 - It can easily expand or shrink as needed.



Linked List-Based Stack Implementation

Stack Operations:

- **push()**: Insert a new element into the stack
 - i.e just insert a new element at the beginning of L .
- **pop()**: Return the top element of the Stack
 - i.e simply return and delete the first element of L .
- **top()**: Return the first element of L .
- **isEmpty()**: returns a boolean value for whether the L is empty or not.

Linked List-Based Stack Implementation

Push operation:

- Initialize a node
- Update the value of that node by data i.e. **node.data = data**
- Insert this node at the head of the linked list
- Update top pointer to the current node

Pop operation:

- First Check whether there is **any node present** in the linked list or not, if not then return
- Otherwise make pointer temp to the top node and move forward the top node by 1 step
- Now free this temp node

```
# Class representing a node in the linked list
class Node:
    def __init__(self, new_data):
        self.data = new_data
        self.next = None
```

```
# Function to push an element onto the stack
def push(self, new_data):

    # Create a new node with given data
    new_node = Node(new_data)

    # Check if memory allocation for the new node failed
    if not new_node:
        print("\nStack Overflow")
        return

    # Link the new node to the current top node
    new_node.next = self.head

    # Update the top to the new node
    self.head = new_node
```

```
# Function to remove the top element from the stack
def pop(self):

    # Check for stack underflow
    if self.is_empty():
        print("\nStack Underflow")
    else:

        # Assign the current top to a temporary variable
        temp = self.head

        # Update the top to the next node
        self.head = self.head.next

        # Deallocate the memory of the old top node
        del temp
```

Linked List-Based Stack Implementation

```
# Class representing a node in the linked list
class Node:
    def __init__(self, new_data):
        self.data = new_data
        self.next = None
```

Top (peek) operation:

- Check if there is any node present or not, if not then return
- Otherwise return the value of top node of the linked list

```
# Function to return the top element of the stack
def peek(self):

    # If stack is not empty, return the top element
    if not self.is_empty():
        return self.head.data
    else:
        print("\nStack is empty")
        return float('-inf')
```

isEmpty operation:

- Check if the head node is none, if not then return False
- Otherwise return True

```
# Function to check if the stack is empty
def is_empty(self):

    # If head is None, the stack is empty
    return self.head is None
```

Linked List-Based Stack Implementation

- **Time Complexity:**
 - Constant time $O(1)$ for all push(), pop(), and peek(), as not performing any kind of traversal over the list.
- **Space Complexity:**
 - The space usage for a stack is $O(n)$.

Feature	Array-Based Stack	Linked-List-Based Stack
Memory	Contiguous, fixed size	Dynamic, grows as needed
Resizing	Costly (requires copying elements)	N/A
Time Complexity	$O(1)$	$O(1)$
Space Complexity	$O(n)$	$O(n)$
Best Use Case	When maximum size is known	When stack size is unpredictable

Reversing Data using a Stack

- As a consequence of the LIFO protocol, a stack can be used as a general tool to reverse a data sequence.
- **Examples:**
 - Printing lines in a file in reverse order.
 - Reversing the elements of a list using a stack.
 - Reversing the order in which elements are stored in a stack.

```
1 # reversing data using a stack
2 def reverse_file(filename):
3     ''' Overwrite given file with its content line-by-line reversed'''
4
5     S = ArrayStack()
6     original = open(filename)
7     for line in original:
8         S.push(line.rstrip('\n')) # we will re-insert newlines when writing
9     original.close()
10
11     # Now we overwrite with contents in LIFO order
12     output = open(filename, 'w') # reopening file overwrites original
13     while not S.is_empty():
14         output.write(S.pop() + '\n') # re-insert newline characters
15     output.close()
16
17 #####
18 file = open("initial.txt", 'w')
19 file.write("I am going home.\n")
20 file.write("Today is a holiday.")
21 file.close()
22
23 !cat initial.txt
24 print('\n\n')
25 reverse_file("initial.txt")
26 !cat initial.txt
```

```
I am going home.
Today is a holiday.
```

```
Today is a holiday.
I am going home.
```

Exercise 5 — Valid pop sequence

- A stack is initially empty.
Push numbers **1, 2, 3, 4, 5** in this order.
- Which of the following **pop sequences** are possible?

A. 5 4 3 2 1

B. 4 5 3 2 1

C. 4 3 5 1 2

D. 3 5 4 2 1

Exercise 6 — How many pushes vs pops?

- A stack is initially empty.
- You execute 20 stack operations.
At the end, the stack contains 4 elements.
- How many **push** operations occurred?

Exercise 6 — How many pushes vs pops?

- A stack is initially empty.
- You execute 20 stack operations.
At the end, the stack contains 4 elements.
- How many **push** operations occurred?

Exercise 7 — Maximum stack size during execution

- Compute the **maximum stack size** during execution:

```
push A
push B
pop
push C
push D
pop
push E
push F
pop
pop
```

Exercise 8 — Function call stack reasoning

- A program executes function calls:

```
main()  
  A()  
    B()  
    C()
```

Write the order in which functions **finish execution**.

Outline

- Stacks
 - Stack Definition
 - Stack Implementation (Array / Linked-list)
 - **Matching Parenthesis**
- Queues
 - Queue Definition
 - Queue Implementation

Matching Parenthesis

It checks for the correct nesting of the parentheses in the expression.

- An expression, i.e., $[(2+3)*x+5]*2$.
- Each open bracket “(”, “{”, or “[” must **be closed with** a matching bracket “)”, “}”, or “]”.
- When you open a bracket, you must close it in the **reverse** order.
- Every close bracket has a corresponding open bracket of the same type.
 - correct:
 1. `( )(( )){([ ( )]}`
 2. `(( ( )(( )){([ ( )]))}`
 - Incorrect:
 1. `)( ( )){([ ( )]}(`
 2. `{([ ])}`
 3. `([ ])`
 4. `(`

Matching Parenthesis

- **Algorithm Steps:**

- Perform a left-to-right scan of the expression using a stack S.
- At reading an opening symbol, push it into the stack.
- At reading a closing symbol, pop and check if the element removed from the top of the stack forms a valid pair with the closing symbol.
- At the end of the scan, if the stack is empty, the expression is properly matched.

- **Time Complexity:**

- For an expression of length n , there are n push and n pop operations.
- Total runtime complexity is **$O(n)$** .

- **Key Points:**

- The set of symbols (delimiters) is fixed (e.g., $()$, $[]$, $\{\}$), making matching operations constant time.

Matching Parenthesis – Example 1

- Input: () (() [()])

i	X[i]	Operation	Stack	Output
0	(	Push (	(	
1	)	Pop (Test if (and X[i] match? YES		
2	(	Push (	(	
3	(	Push (	((	
4	)	Pop (Test if (and X[i] match? YES	(	
5	[	Push [	([	

Matching Parenthesis – Example 1

- Input: () (() [()])

i	X[i]	Operation	Stack	Output
6	(	Push (	([(	
7	)	Pop (Test if (and X[i] match? YES	([	
8	]	Pop [Test if [and X[i] match? YES	(	
9	)	Pop (Test if (and X[i] match? YES		
		Test if stack is Empty? YES		TRUE

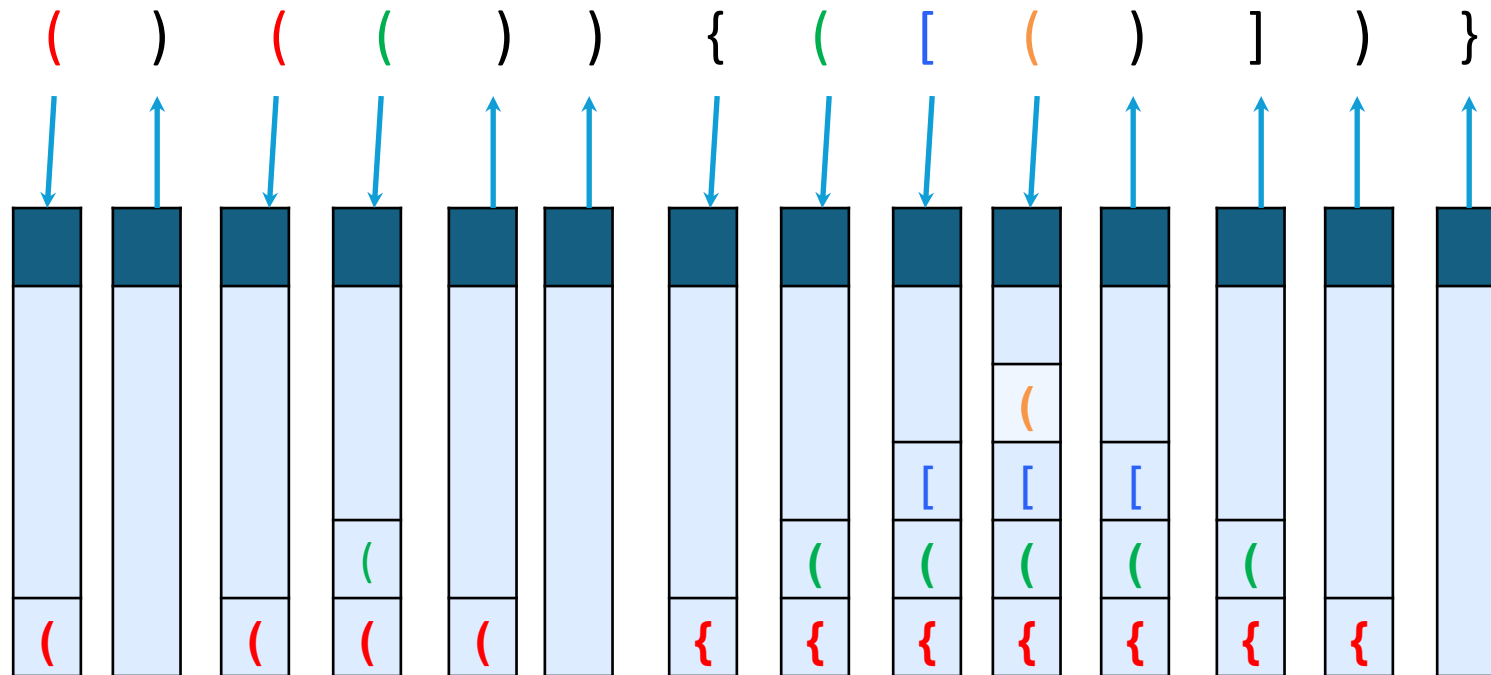
Matching Parenthesis – Example 2

- Input: (() [] ()

i	X[i]	Operation	Stack	Output
0	(	Push (	(	
1	(	Push (	((	
2	)	Pop (Test if (and X[i] match? YES	(	
3	[	Push [	([	
4	]	Pop [Test if [and X[i] match? YES	(	
5	)	Pop (Test if (and X[i] match ? NO		FASLE

Matching Parenthesis

Check the following parentheses expression:



Matching Parenthesis – Algorithm

Algorithm ParenMatch(X, n):

Input: An array X of n tokens, each of which is either a grouping symbol, a variable, an arithmetic operator, or a number

Output: **true** if and only if all the grouping symbols in X match

Let S be an empty stack

for $i=0$ to $n-1$ **do**

if $X[i]$ is an opening grouping symbol **then**

$S.push(X[i])$

else if $X[i]$ is a closing grouping symbol **then**

if $S.isEmpty()$ **then**

return false {nothing to match with}

if $S.pop()$ does not match the type of $X[i]$ **then**

return false {wrong type}

if $S.isEmpty()$ **then**

return true {every symbol matched}

else

return false {some symbols were never matched}

Exercise 9 — Parentheses matching (answer)

- Determine if each is valid:

({ [] })

([])

((()))

{ [() ()] }

(((

Exercise 10 — Min Stack

- Design a stack supporting:

```
push(x)  
pop()  
top()  
getMin() -> O(1)
```

- Example:

```
push(3)  
push(5)  
push(2)  
getMin() -> 2  
pop()  
getMin() -> 3
```

Hint: use an **auxiliary stack**.

Outline

- Stacks
 - Stack Definition
 - Stack Implementation (Array / Linked-list)
 - Matching Parenthesis
- Queues
 - **Queue Definition**
 - Queue Implementation

Queues - Example

Imagine you and your friends are going to buy bubble tea, and there's already **a long line in front of the store**. What would you do?

1. Follow the rules and **join the end of the line**, waiting for everyone ahead of you to get their bubble tea before it's your turn to order.
2. Cut in line and **go straight to the front** to place your order.

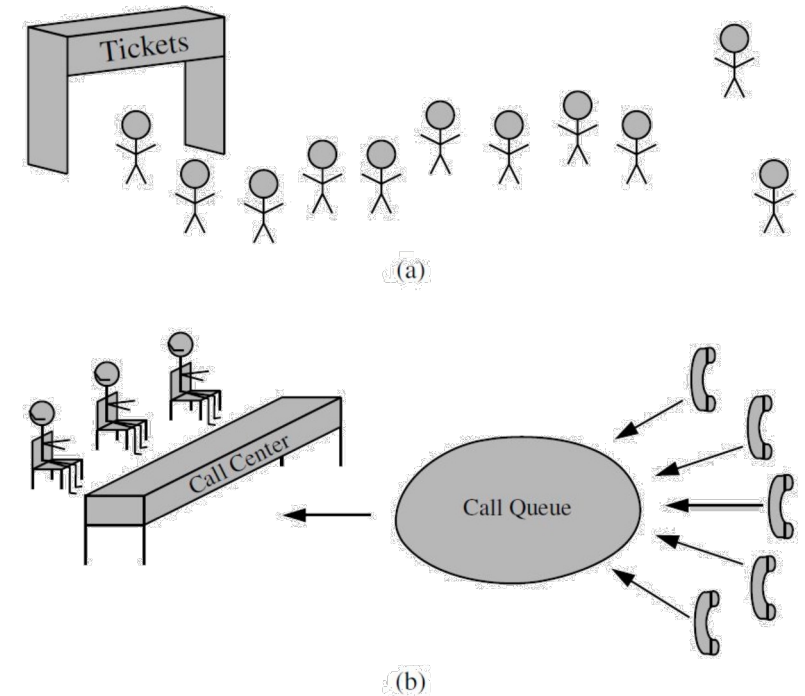
You would likely choose the first approach, because waiting in line follows the principle of "**first come, first served**." The person who arrives first gets served first, ensuring fairness and order.

This "**First-In, First-Out**" (FIFO) characteristic is the essence of the **Queue** data structure!



Queues

- It is a close “cousin” of the stack.
- A **queue** is a collection of objects that are inserted and removed according to the **first-in, first-out (FIFO)** principle.
- The elements can be inserted at any time, but only the element that has been in the queue the longest can be next removed.
- Examples:
 - People waiting to get on an amusement park ride.
 - Phone calls being routed to a customer service center.



The Queue Abstract Data Type

- A queue is an abstract data type that follows the **First-In-First-Out** (FIFO) principle. It supports the following operations:
- The queue ADT supports two fundamental methods for a queue Q:
 - **enqueue (e)** : Add element e to the back of Q.
 - **dequeue ()** : Remove and return the **least recently inserted (front)** element from Q.
- Additional methods:
 - **first ()** : Return the element at the front of queue Q, **without removing it**;
 - **is_empty ()** : Return **True** if queue Q does not contain any elements.
 - **len (Q)** : Return the number of elements in queue Q.

The Queue Abstract Data Type

- By convention, we assume that a newly created queue is empty.
- There is no a priori bound on the capacity of the queue.
- Elements added to the queue can have arbitrary type.

Operation	Return Value	first $\leftarrow$ Q $\leftarrow$ last
Q.enqueue(5)	—	[5]
Q.enqueue(3)	—	[5, 3]
len(Q)	2	[5, 3]
Q.dequeue()	5	[3]
Q.is_empty()	False	[3]
Q.dequeue()	3	[]
Q.is_empty()	True	[]
Q.dequeue()	“error”	[]
Q.enqueue(7)	—	[7]
Q.enqueue(9)	—	[7, 9]
Q.first()	7	[7, 9]
Q.enqueue(4)	—	[7, 9, 4]
len(Q)	3	[7, 9, 4]
Q.dequeue()	7	[9, 4]

Exercise 1 — Basic FIFO Trace

- Start with an empty queue. Perform:

```
enqueue (10)
enqueue (20)
dequeue ()
enqueue (30)
dequeue ()
enqueue (40)
enqueue (50)
dequeue ()
```

Exercise 2 — Operation Counting

- A queue performs 25 operations.
At the end, it contains 7 elements.
- How many enqueues and dequeues occurred?

Exercise 3 — Queue Size Over Time

```
enqueue A  
enqueue B  
enqueue C  
dequeue  
dequeue  
enqueue D  
enqueue E  
enqueue F  
dequeue
```

- Maximum size?
- Final queue?

Outline

- Stacks
 - Stack Definition
 - Stack Implementation (Array / Linked-list)
 - Matching Parenthesis
- Queues
 - Queue Definition
 - **Queue Implementation**

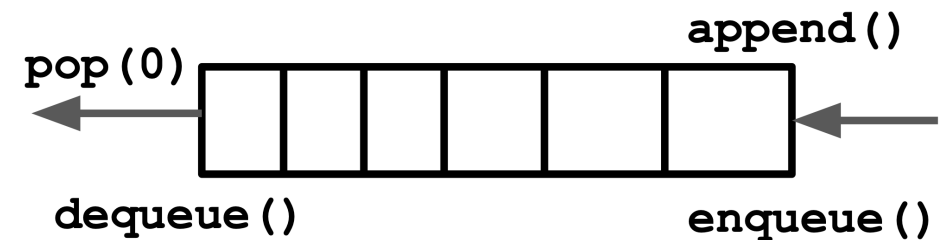
Linked-List-based Implementation

- Store the elements of S in a linked list L .
- En-queue(e): insert e at the end of L .
- De-queue: delete the head node of L and return the element therein.
- At all times, keep track of the addresses of the head and the tail.
- Guarantees:
 - $\mathbf{O}(n)$ space, *where* $n = |S|$ (assuming each element in S occupies $O(1)$ memory).
 - En-queue in $\mathbf{O}(1)$ time.
 - De-queue in $\mathbf{O}(1)$ time.

Array-based Queue Implementation (1)

Implementation #1

- Use `append(e)` to add element `e` at the end of the array.
- Use `pop(0)` to remove the first element from the list when dequeuing.
 - Inefficient implementation with $\Theta(n)$ time.



Array-based Queue Implementation (2)

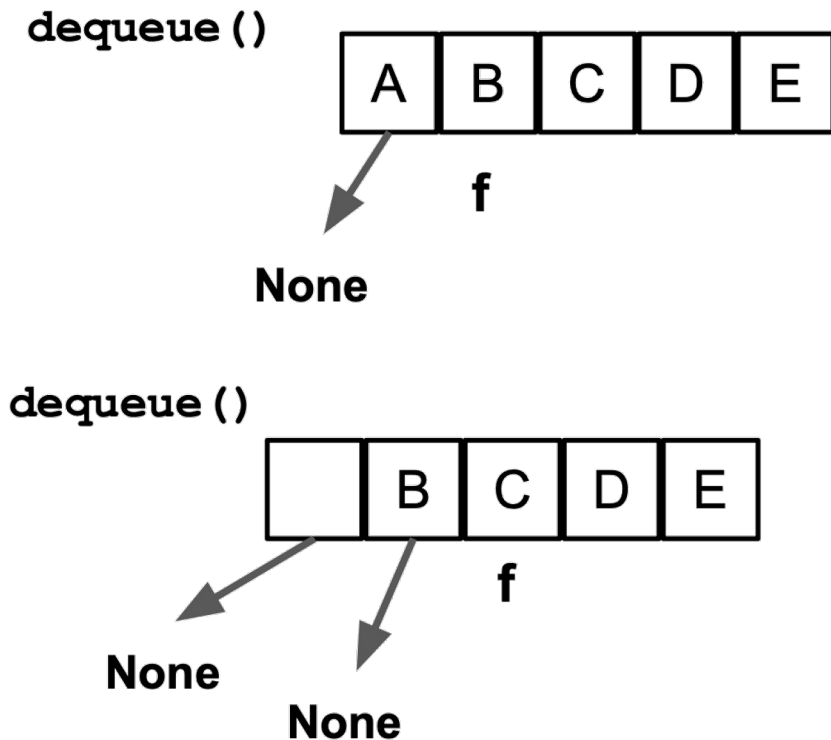


Implementation #2

- Let **m** be the total number of enqueue operations.
- Allocate an array with size m.
- Same method for adding elements.
 - Maintain an explicit variable f to store the index of the element that is currently at the front of the queue.
 - Replace the dequeued entry in the array with a reference to None.
 - **En-queue and De-queue time: $O(1)$.**
 - **Space: $O(m)$,** even if the actual number of elements n is smaller ($m > n$). This means the queue may occupy more space than needed.

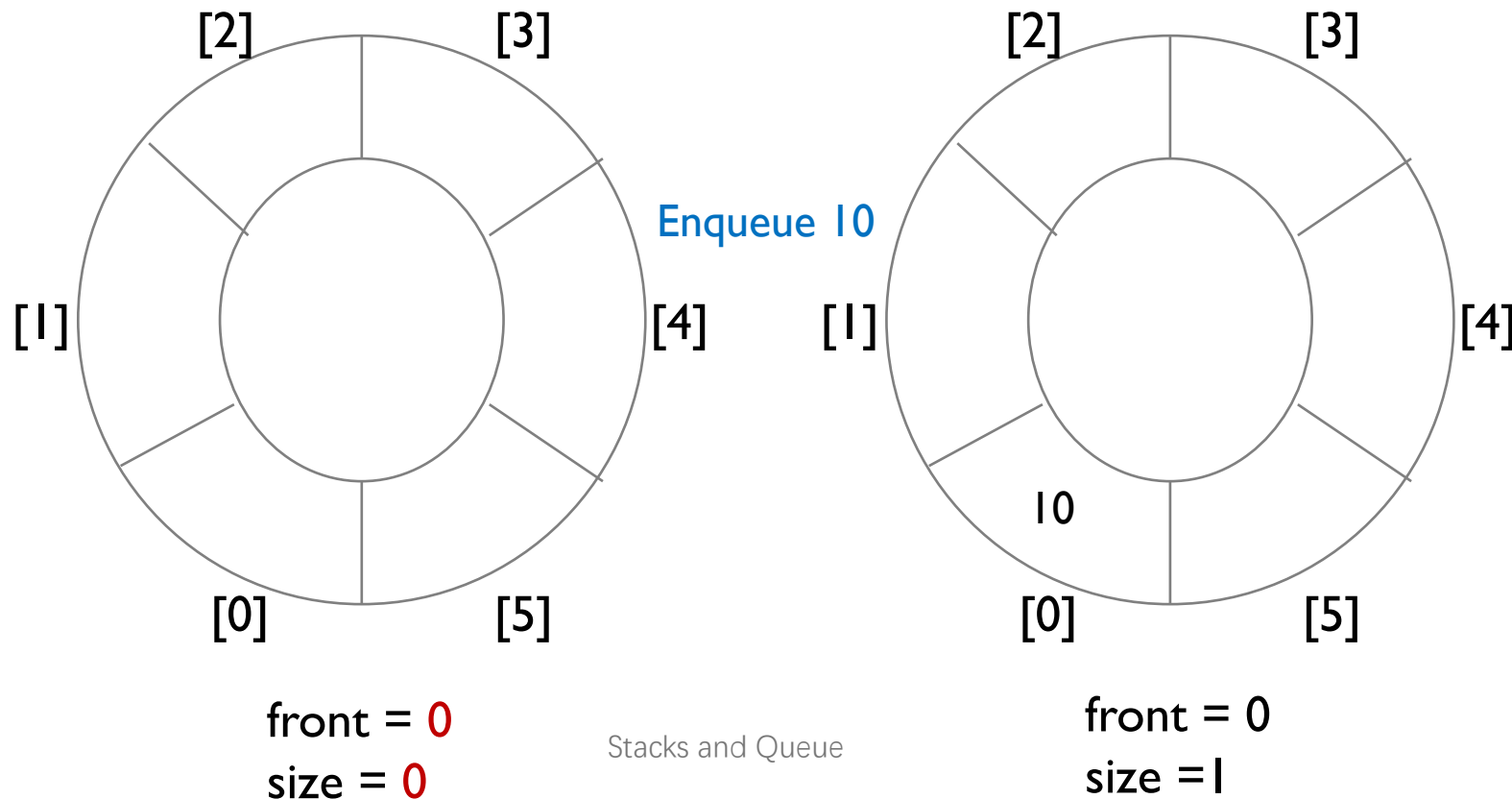
This design would have detrimental consequences in applications in which queues have relatively modest size, but which are used for long periods of time.

For example, the wait-list for a restaurant might never have more than 30 entries at one time, but over the course of a day (or a week), the overall number of entries would be significantly larger.

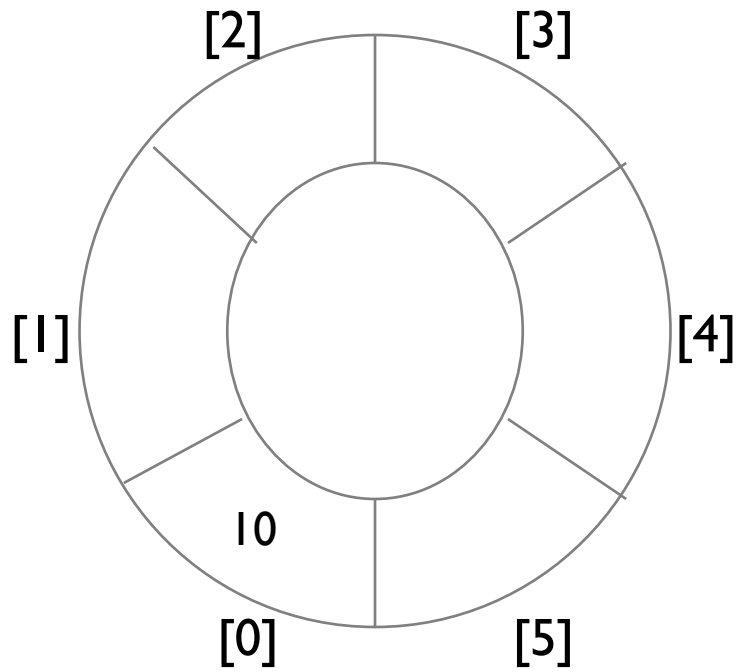


Using an Array Circularly

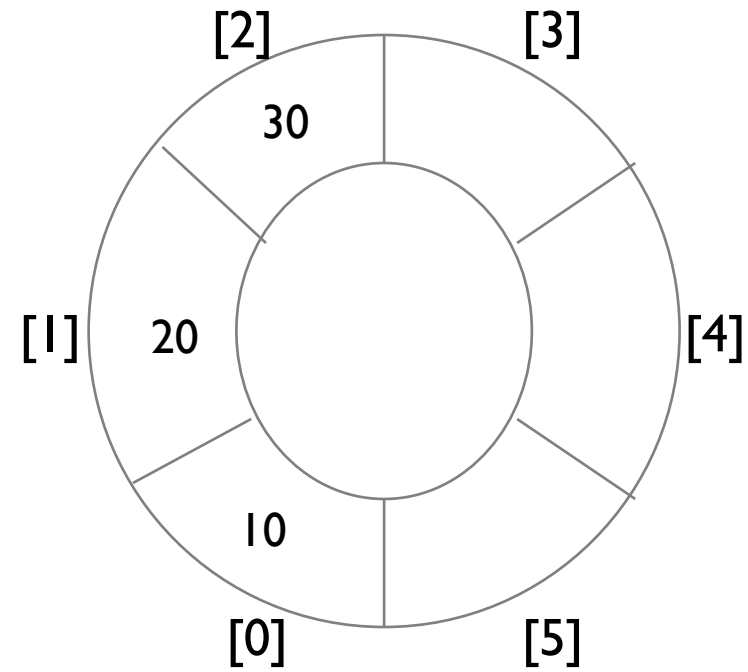
This implementation uses a **fixed-size array** of length **N** (where N is larger than the number of elements in the queue) and allows the queue to **wrap around** the array:



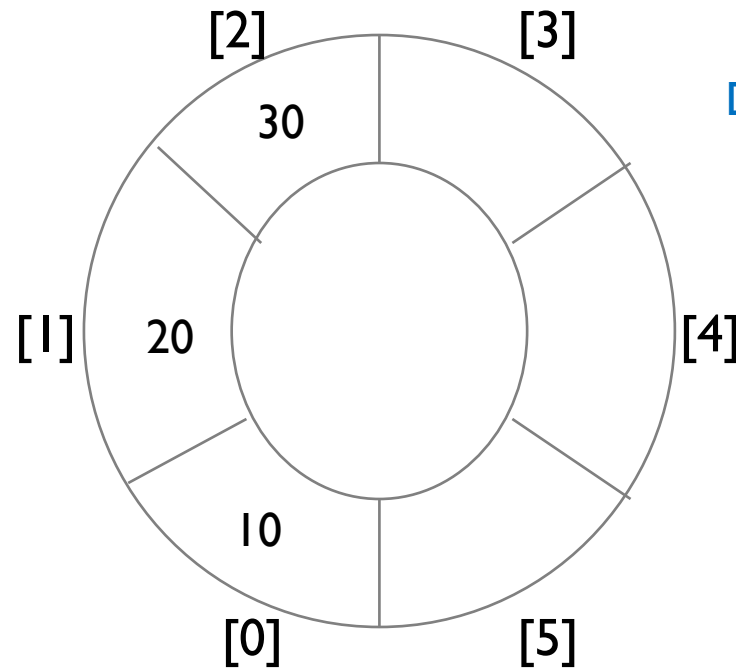
Enqueue 20; Enqueue 30



front = 0
size = 1

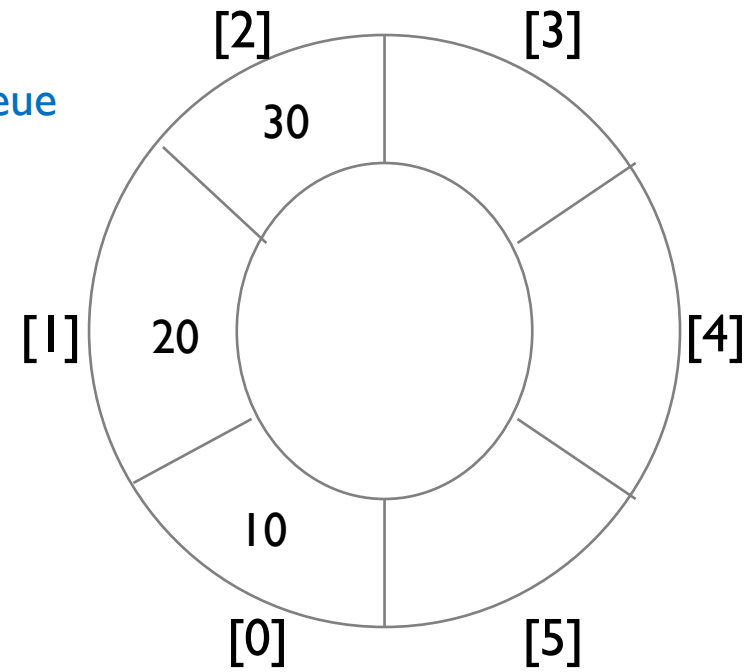


front = 0
size = 3



front = 0
size = 3

De-queue

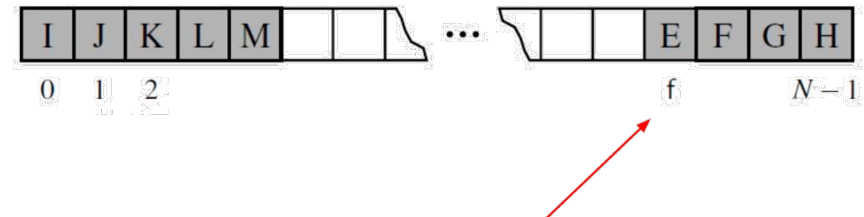


front = 1
size = 2

Using an Array Circularly

This implementation uses a **fixed-size array** of length **N** (where N is larger than the number of elements in the queue) and allows the queue to **wrap around** the array:

1. The **front** of the queue can move rightward, and elements can wrap around from the end of the array to the start.
2. New elements are added at the "end" of the queue, progressing from the front to index **N-1** and then continuing at index **0**.
3. This approach ensures **O(1)** De-queue/En-queue time by reducing the cost of moving elements.



A Python Queue Implementation

- The **queue class** uses three instance variables:
 - **_data**: A fixed-capacity list storing the queue elements.
 - **_size**: The current number of elements in the queue.
 - **_front**: The index of the first element in the queue.
- **Accessor methods**:
 - **len()**: Returns the current queue length.
 - **is_empty()**: Checks if the queue is empty.
 - **first()**: Returns the front element of the queue.
- **Mutator methods**:
 - **enqueue()**: Adds an element to the queue.
 - **dequeue()**: Removes an element from the queue.

```
1 class ArrayQueue:
2     '''
3     FIFO Queue implementation using a Python List as underlying storage
4     '''
5     DEFAULT_CAPACITY = 5      # moderate capacity for all new queues
6
7     def __init__(self):
8         ''' Create an empty queue '''
9         self._data = [None] * ArrayQueue.DEFAULT_CAPACITY
10        self._size = 0
11        self._front = 0
12
13
14    def __len__(self):
15        ''' return the number of elements in the queue'''
16        return self._size
17
18    def is_empty(self):
19        ''' Return True if the queue is empty'''
20        return self._size == 0
21
22    def first(self):
23        '''
24        Return (but do not remove) the element at the front of the queue
25        Raise Empty Exception if the queue is empty.
26        '''
27        if self.is_empty():
28            raise Empty('Queue is Empty')
29        return self._data[self._front]
```

```

32 def dequeue(self):
33     '''
34     remove and return the first element of the queue.
35     raise Empty exception if the queue is empty.
36     '''
37     if self.is_empty():
38         raise Empty('Queue is Empty')
39     answer = self._data[self._front]
40     self._data[self._front] = None # help garbage collection
41     self._front = (self._front + 1) % len(self._data) # circular indexing
42     self._size -= 1 # reduce the queue size
43     return answer
44
45 def enqueue(self, e):
46     '''Add an element to the back of queue'''
47     if self._size == len(self._data):
48         self._resize(2*len(self._data)) # double the array size
49     avail = (self._front + self._size) % len(self._data)
50     self._data[avail] = e
51     self._size += 1
52
53
54 def _resize(self, cap):
55     '''resize to a new list of capacity >= len(self)'''
56     old = self._data
57     self._data = [None] * cap
58     walk = self._front
59     for k in range(self._size): # only consider existing elements
60         self._data[k] = old[walk] # intentionally shift indices
61         walk = (1+walk) % len(old) # use old size as modulus
62     self._front = 0 # front has been realigned.
63
64 def __str__(self):
65     '''string representation of the queue'''
66     return '<'+''.join(str(self._data))+'>'

```

```

69 #####
70
71 Q = ArrayQueue()
72 Q.enqueue(5)
73 Q.enqueue(7)
74 Q.enqueue(9)
75 Q.enqueue(2)
76 Q.enqueue(6)
77 Q.enqueue(4)
78 Q.enqueue(1)
79 Q.enqueue(0)
80 |
81 print('Q: ', Q)
82 print('Queue Length:', len(Q))
83 print('Remove last item: ', Q.dequeue())
84 print('Remove last item: ', Q.dequeue())
85 print('Q: ', Q)
86 print('Queue Length:', len(Q))
87
88
89

```

Q: <[5, 7, 9, 2, 6, 4, 1, 0, None, None]>

Queue Length: 8

Remove last item: 5

Remove last item: 7

Q: <[None, None, 9, 2, 6, 4, 1, 0, None, None]>

Queue Length: 6

Adding and Removing Elements

- Index where the next element is added to the queue:

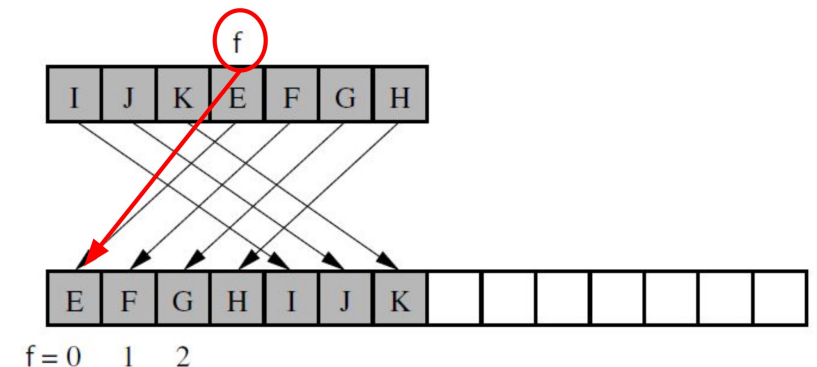
`avail = (self._front + self._size) % len(self._data)`

- During dequeuing, the value at index `self._front` is returned and removed. The `_front` index is updated to the next position in the circular queue:

`self._front = (self._front + 1) % len(self._data)`

Resizing the Queue

- When queue size equals the size of the underlying array, the storage capacity of the underlying list is doubled.
- The references from the old list are copied into the new list.
- While transferring the contents, we intentionally realign the front of the queue with index 0 in the new array.



Shrinking the Underlying Array

- A robust approach is to reduce the array to half of its current size, whenever the number of elements stored in it falls below one fourth of its capacity.
- Space usage: $\sim \Theta(n)$

```
def dequeue(self):
    """
    remove and return the first element of the queue.
    raise Empty exception if the queue is empty.
    """
    if self.is_empty():
        raise Empty('Queue is Empty')
    answer = self._data[self._front]
    self._data[self._front] = None # help garbage collection
    self._front = (self._front + 1) % len(self._data) # circular indexing
    self._size -= 1 # reduce the queue size

    if 0 < self._size < len(self._data) // 4: # shrink the array size by half
        self._resize(len(self._data)//2) # when queue size 1/4 of the
    return answer # total array capacity
```

```
75 Q = ArrayQueue()
76 Q.enqueue(5)
77 Q.enqueue(7)
78 Q.enqueue(9)
79 Q.enqueue(2)
80 Q.enqueue(6)
81 Q.enqueue(4)
82 Q.enqueue(1)
83 Q.enqueue(0)
84
85 print('Q: ', Q)
86 print('Queue Length:', len(Q))
87 print('Remove last item: ', Q.dequeue())
88 print('Remove last item: ', Q.dequeue())
89 print('Q: ', Q)
90 print('Queue Length:', len(Q))
91 print('Remove last item: ', Q.dequeue())
92 print('Remove last item: ', Q.dequeue())
93 print('Remove last item: ', Q.dequeue())
94 print('Remove last item: ', Q.dequeue())
95 print('Q: ', Q)
96 print('Queue Length:', len(Q))
97 print('Remove last item: ', Q.dequeue())
98 print('Q: ', Q)
99 print('Queue Length:', len(Q))
100
101
102
```

```
Q:  <[5, 7, 9, 2, 6, 4, 1, 0, None, None]<
Queue Length: 8
Remove last item: 5
Remove last item: 7
Q:  <[None, None, 9, 2, 6, 4, 1, 0, None, None]<
Queue Length: 6
Remove last item: 9
Remove last item: 2
Remove last item: 6
Remove last item: 4
Q:  <[None, None, None, None, None, None, 1, 0, None, None]<
Queue Length: 2
Remove last item: 1
Q:  <[0, None, None, None, None]<
Queue Length: 1
```

Analyzing the Array-Based Queue Implementation

Operation	Running Time
Q.enqueue(e)	$O(1)^*$
Q.dequeue()	$O(1)^*$
Q.first()	$O(1)$
Q.is_empty()	$O(1)$
len(Q)	$O(1)$

*amortized

Exercise 4 — Circular Queue Indexing

- Capacity = 5
Initially: front = 0, size = 0
- Operations:

```
enqueue 10  
enqueue 20  
enqueue 30  
dequeue  
dequeue  
enqueue 40  
enqueue 50  
enqueue 60
```

At what index is 60 inserted?

Exercise 5 — Queue Using Two Stacks

- Design queue using two stacks.

Exercise 6 — Simulate mixed operations

- You have: Stack S, Queue Q
- Both start empty.
- Perform operations:

```
push (S, 1)
push (S, 2)
enqueue (Q, 3)
enqueue (Q, 4)
pop (S) → enqueue (Q, ?)
dequeue (Q) → push (S, ?)
push (S, 5)
enqueue (Q, 6)
pop (S) → enqueue (Q, ?)
```

How S and Q looks like finally?

Exercise 7 — Check if stack permutation possible

- Input queue: [1,2,3,4]
Target output: [2,1,4,3]
- Can we generate this using **one stack**?

Exercise 8 — Implement stack using two queues

- Design stack operations using **two queues**.

Push(x):

Pop():

Top():

Exercise 9 — Interleave queue halves

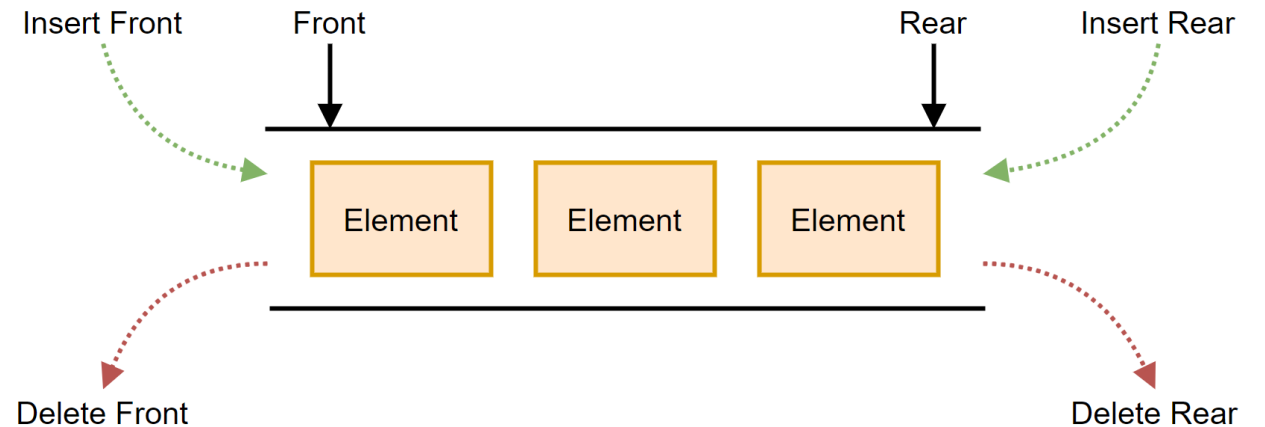
- Queue: [1,2,3,4,5,6]
- Rearrange $\rightarrow$ [1,4,2,5,3,6] using stack + queue.

Exercise 10 — Evaluate postfix expression

- Evaluate using stack:
- $3\ 4\ +\ 2\ *\ 7\ /$

Double-ended queue (Extended)

- A **Deque** (Double-Ended Queue) allows insertion and deletion at **both ends**.
 - Combines features of **stack (LIFO)** and **queue (FIFO)**.
- **Core Operations**
 - `insertFront(x) / insertRear(x)`
 - `deleteFront() / deleteRear()`
 - `peekFront() / peekRear()`
 - `isEmpty(), size()`
- **Typical Implementations**
 - Circular array (ring buffer)
 - Doubly linked list



Sliding Window Maximum (Extended)

- Given an array and a window size **k**:
 - A “window” slides from left $\rightarrow$ right.
- For each position, output the **maximum value inside the window**.

Example:

```
nums = [2, 1, 3, 4, 6, 3, 8, 9, 10, 12, 56]
```

```
k = 4
```

Windows and max:

```
[2 1 3 4] 6 3 8 9 10 12 56  $\rightarrow$  4
```

```
2 [1 3 4 6] 3 8 9 10 12 56  $\rightarrow$  6
```

```
2 1 [3 4 6 3] 8 9 10 12 56  $\rightarrow$  6
```

```
2 1 3 [4 6 3 8] 9 10 12 56  $\rightarrow$  8
```

```
2 1 3 4 [6 3 8 9] 10 12 56  $\rightarrow$  9
```

```
...
```

Output:

```
[4, 6, 6, 8, 9, 10, 12, 56]
```

Sliding Window Maximum (Extended)

- `nums = [1, 3, -1, -3, 5, 3, 6, 7]`
- `k = 3`
- Goal: for each window of size 3, output the maximum.

Step 1 — Remove leaving element

Step 2 — Remove smaller elements from the back

Step 3 — Add new elements to back

Step 4 — Front = maximum of window

Step-by-Step Sliding Window (Extended)

- **$i = 0$, value = 1 :**

- Window not full yet.

`window = [1]`

`dq = [1]`

- **$i = 1$, value = 3 :**

- Remove smaller from back:
- $1 < 3 \rightarrow$ remove 1

`window = [1, 3]`

`dq = [3]`

- **$i = 2$, value = -1:**

- No removal from back
- $(3 > -1)$

`window = [1, 3, -1]`

`dq = [3, -1]`

`max = 3`

- **$i = 3$, value = -3**

- Element leaving window = 1;
- $1 \neq dq[0] \rightarrow$ no removal;
- No removal from back $(-1 > -3)$

`window = [3, -1, -3]`

`dq = [3, -1, -3]`

`max = 3`

Step-by-Step Sliding Window (Extended)

- **i = 4, value = 5**
 - Element leaving = 3
3 == dq[0] → remove front
 - Now remove smaller from back:
 - -3 < 5 → remove
 - -1 < 5 → remove

```
window = [-1, -3, 5]
dq = [5]
max = 5
```

- **i = 5, value = 3**
 - Element leaving = -1
 - -1 ≠ dq[0]
 - Remove smaller from back:
 - 5 > 3 → keep

```
window = [-3, 5, 3]
dq = [5, 3]
max = 5
```

- **i = 6, value = 6**
 - Element leaving = -3
 - -3 ≠ dq[0]
 - Remove smaller from back:
 - 3 < 6 → remove
 - 5 < 6 → remove

```
window = [5, 3, 6]
dq = [6]
max = 6
```

- **i = 7, value = 7**
 - Element leaving = 5
 - 5 ≠ dq[0]
 - Remove smaller from back:
 - 6 < 7 → remove

```
window = [3, 6, 7]
dq = [7]
max = 7
```

Exercise 11 — Sliding Window Maximum

```
nums = [4, 2, 2, 6, 1, 6, 3, 5]  
k = 3  
dq = []
```

For each position, output the **maximum value inside the window**.
And show the content of dq